

Introductory Econometrics with Recitation: TA Session Week 5

Danny Po-Hsien Kang (康柏賢)

March 24, 2025

Today's agenda

- 1 Remove everything in the environment
 - `rm(list = ls())`
- 2 Assign Column Names
 - `colnames()`
- 3 Data Wrangling
 - `na.omit()`
 - `as.numeric()`
- 4 Linear Regression: Estimation
 - `cor()`
 - `lm()`
 - `summary()`
- 5 Data Visualization
 - `geom_point()`
 - `scale_x_continuous(), scale_y_continuous()`
 - `geom_abline()`

Today's Dataset

- Today we will use the following CSV file from the OpenIntro Website:
 - `ncbirths.csv`: A 2004 dataset from North Carolina on births, including mothers' habits and practices, with 1,000 cases.
- Please import the following datasets into RStudio.

```
birth <- read.csv("data/ncbirths.csv")
```

Remove Everything: `rm(list = ls())`

- `rm(list = ls())` removes everything inside the global environment.
 - Useful when you want to re-run the whole code.
- `ls()` returns a character vector of object names in the current environment.

```
x <- 1
y <- "hi"
ls()
# [1] "x" "y"
```

Assign Column Names: colnames()

- **Syntax:**

```
colnames(df) <- new_names
```

- **Example:**

```
df <- data.frame(a = 1:3, b = 4:6)  
colnames(df) <- c("height", "weight")
```

- **Output:**

	height	weight
1	1	4
2	2	5
3	3	6

Assign Column Names: colnames()

- colnames() works on matrices too.
- **Example:**

```
m <- matrix(1:6, nrow = 3)
colnames(m) <- c("col1", "col2")
```

- **Output:**

	col1	col2
[1,]	1	4
[2,]	2	5
[3,]	3	6

Remove Missing Values: na.omit()

- na.omit() removes NAs in vectors or dataframes.

- **Syntax:**

```
data_clean <- na.omit(data)
```

- **Example (vector):**

```
x <- c(1, NA, 3, NA, 5)
x_clean <- na.omit(x)
print(x_clean)
```

- **Output:**

```
# keeps an attribute (extra information attached to object)
[1] 1 3 5
attr("na.action") # you can use as.vector() to remove these
[1] 2 4
attr("class")
[1] "omit"
```

Remove Missing Values: na.omit()

- **Example (dataframe):**

```
df <- data.frame(  
  x = c(1, NA, 3, 4, NA),  
  y = c(10, 20, NA, 40, 50)  
) %>%  
  na.omit()  
print(df)
```

- **Output:**

```
  x  y  
1 1 10  
4 4 40
```

- Sometimes using na.omit() may be too aggressive...

Create Numeric Type Object: `as.numeric()`

- **Syntax:**

```
data_numeric <- as.numeric(vector)
```

- **Example:**

```
logical_to_num <- as.numeric(c(TRUE, FALSE, TRUE))  
character_to_num <- as.numeric(c("0", "1.1", "-6.5"))
```

- **Output**

```
> print(logical_to_num)  
[1] 1 0 1  
  
> print(character_to_num)  
[1] 0.0 1.1 -6.5
```

Create Numeric Type Object: `as.numeric()`

- Lists and data.frame can not be converted by `as.numeric()` directly.
- `as.numeric("Hello World!")` gives NA.
- Similar (and useful) functions: `as.logical()`, `as.integer()`, `as.character()`
- `as.numeric()` is a powerful tool for data cleaning/analysis.

```
data_example <- data.frame(name = c("Alice", "Bob", "Cindy"),
                           score = c("80", "60", "100")
                           )
mean(data_example$score) # NA !
mean(as.numeric(data_example$score)) # 80

data_example$score <- as.numeric(data_example$score)
mean(data_example$score) #80
```

Correlation Matrix: cor()

- **Syntax:**

```
cor(data$var1, data$var2)
```

- **Example:**

```
cor(birth$weeks, birth$weight)
```

- **Output:**

```
[1] 0.6358891
```

Correlation Matrix: cor()

- **Syntax:**

```
data %>%  
  select(var1, var2, ..., varN) %>%  
  cor()
```

- **Example:**

```
birth %>%  
  na.omit() %>%  
  select(mage, gained, weeks, weight) %>%  
  cor()
```

Correlation Matrix: cor()

- Output:

	mage	gained	weeks	weight
mage	1.0000000	-0.05948610	-0.04000760	0.0516980
gained	-0.0594861	1.00000000	0.09857971	0.1660534
weeks	-0.0400076	0.09857971	1.00000000	0.6358891
weight	0.0516980	0.16605345	0.63588911	1.0000000

Fit Linear Model: `lm()`

- **Syntax:**

```
lm_model <- lm(data = myData, Y ~ X)
```

- **Example:**

```
lm_model <- lm(data = birth, weight ~ weeks)  
print(lm_model)
```

- **Output:**

```
> print(lm_model)  
  
Call:  
lm(formula = weight ~ weeks, data = birth)  
  
Coefficients:  
(Intercept)          weeks  
    -6.0953         0.3443
```

Fit Linear Model: `lm()`

- We may also run a multiple linear regression using `lm()`.
- **Syntax:**

```
lm_model <- lm(data = myData, Y ~ X_1 + X_2 + X_3)
```

- **Example:**

```
lm_model_multiple <- lm(
  data = birth,
  weight ~ weeks + gender + visits)
```

- **Output:**

Call:

```
lm(formula = weight ~ weeks + gender + visits, data = birth)
```

Coefficients:

(Intercept)	weeks	gendermale	visits
-6.208828	0.339617	0.365473	0.009322

Obtain Regression Result: `summary()`

• Syntax

```
summary(lm_model)
```

- `lm()` fits the linear model and returns a model object with raw ingredients.
 - e.g. coefficients, residuals, fitted values ...
- `summary()` analyzes and reports that fit.
 - It computes the summary statistics most people want to read.
 - e.g. t-value, r-squared ... these are not saved or computed in `lm()`.
- Thus, we usually use `summary()` to report the results from `lm()`.
 - Also for `aov()`, `glm()`, etc.
- Similar to `t.test()`, `summary()` and `lm()` save the results as lists.

Obtain Regression Result: summary()

● Output

```
Call:
lm(formula = weight ~ weeks, data = birth)

Residuals:
    Min       1Q   Median       3Q      Max
-3.5775 -0.7048 -0.0235  0.7022  4.4165

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) -6.09529    0.46464  -13.12  <2e-16 ***
weeks        0.34433    0.01209   28.49  <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.119 on 996 degrees of freedom
Multiple R-squared:  0.449, Adjusted R-squared:  0.4485
F-statistic: 811.7 on 1 and 996 DF, p-value: < 2.2e-16
```

Scatter Plot: `geom_point()`

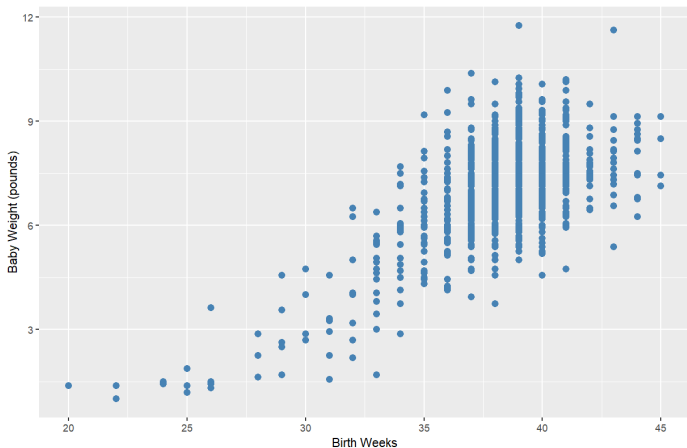


Figure: Scatter plot of multiple unit ratio and home ownerships.

Scatter Plot: `geom_point()`

- **Syntax:**¹

```
ggplot(data, aes(x = ..., y = ...)) +  
  geom_point(color = ...,  
             size = ...,  
             shape = ...  
            )
```

- **Example:**²³

```
ggplot(birth, aes(x = weeks, y = weight)) +  
  geom_point(color = "steelblue", size = 2.5) +  
  labs(  
    x = "Birth Weeks",  
    y = "Baby Weight (pounds)"  
  )
```

¹There are more options in the function, see [here](#).

²R color [cheatsheet](#).

³ggplot2 [Quick Reference](#): shape

Adjust Scale of x, y Axes: `scale_x_continuous()`

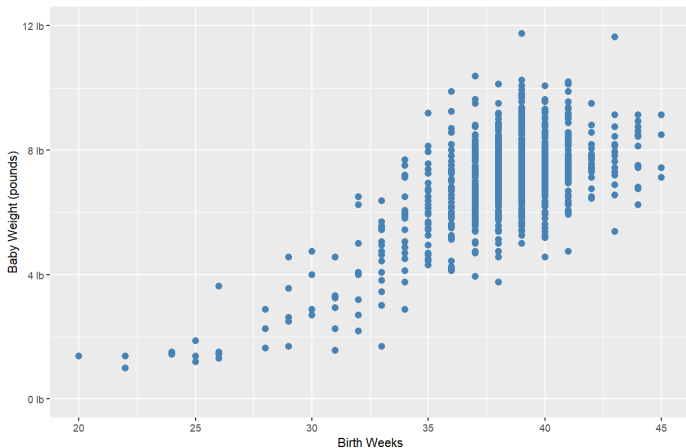


Figure: Scatter plot with adjusted scale.

Adjust Scale of x, y Axes: scale_x_continuous()

● Syntax:

```
Your_Plot +  
  scale_x_continuous(breaks = ..., labels = ..., limits = ...)  
  scale_y_continuous(breaks = ..., labels = ..., limits = ...)
```

● Example:

```
.. +  
  scale_x_continuous(  
    limits = c(20, 45),  
    breaks = c(20, 25, 30, 35, 40, 45)  
  ) +  
  scale_y_continuous(  
    limits = c(0, 12),  
    breaks = c(0, 4, 8, 12),  
    labels = c("0 lb", "4 lb", "8 lb", "12 lb")  
  )
```

Fitted Line in Graph: `geom_abline()`

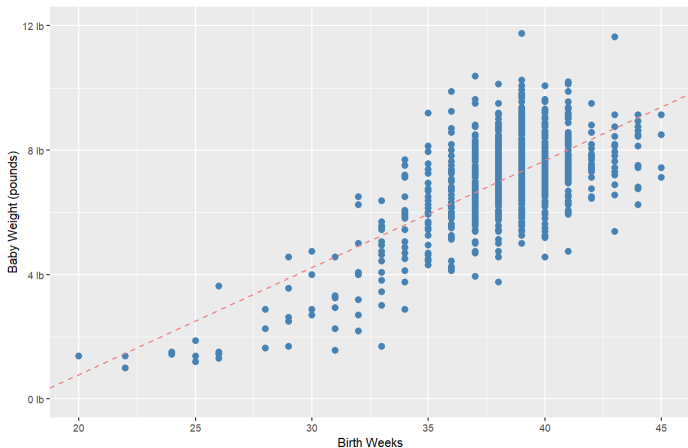


Figure: Scatter plot of weeks and weight with fitted line.

Fitted Line in Graph: geom_abline()

- **Syntax:**

```
Your_Plot +  
  geom_abline(slope = ..., intercept = ...)
```

- **Example:**

```
# obtain the estimated coefficients  
intercept <- summary(lm_model)$coefficient[1]  
slope <- summary(lm_model)$coefficient[2]  
  
# add fitted line  
Your_Plot +  
  geom_abline(  
    slope = slope,  
    intercept = intercept,  
    color = "lightcoral",  
    linetype = "dashed" # dotted, solid...  
  )
```